CGL-sheet13

Robin Kühnberger[1] and Felix Naraschewski[2]

[1]University Stuttgart
[2]University Stuttgart

January 24, 2026

Contents

1 Berechnung der Eulerschen Zahl mit C++ 1

2 Vektoren I 2

3 Vektoren II 3

4 Vektoren III 3

1 Berechnung der Eulerschen Zahl mit C++

Für die Funktion `fakultaet(int k)` ist ein `long` bzw. `unsigned long` als Rückgabedatentyp sinvoll, da $n!$ schnell wächst und die $Wertemenge \in \mathbb{N}$ ist. Aber `approximate_e(int n)` kann solche großen Integer nicht verarbeiten, da es nur ein `double` zurückgeben kann, da $e \in \mathbb{R}$

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
// Funktion zur Berechnung der Fakultaet k!
unsigned long fakultaet(int k)
{
    unsigned long result = 1;
    for (int i = 1; i <= k; i++)
    {
        result *= i;
    }
```

```
    return result;
}
// Funktion zur Annaeherung von e durch n Terme
double approximate_e(int n)
{
    double summe = 0.0;
    for (int k = 0; k < n; ++k)
    {
        summe += (1.0 / static_cast<double>(fakultaet(k)));
        // cout << summe << endl;
        //  --- HIER CODE ERGÄNZEN: Addiere 1.0 / k! zur Summe ---
        //  Hinweis: Nutze die Funktion fakultaet(k)
    }
    return summe;
}
int main()
{
    int iterationen = 20;
    // Berechne e
    double e_approx = approximate_e(iterationen);
    // Ausgabe des Ergebnisses
    cout << "Annaeherung von e nach " << iterationen << " Iterationen:"
    ↪  << endl;
    cout << setprecision(8) << e_approx << endl;
    // --- HIER CODE ERGÄNZEN: Gib e_approx mit 8 Nachkommastellen aus
    ↪  ---
    // Tipp: Nutze setprecision aus <iomanip>
    return 0;
}
```

2 Vektoren I

Die Übergabe als `const vector<double> &vector` ist sinnvoll, da zum einen `const` sicherstellt, dass die Funktion den Vektor nicht verändern kann, da die Funktion nur den Vektor ausließt. Und zum anderen wird bei sehr großen Vektoren Zeit und Speicher gespahrt, da der Verktor nich koppiert sondern nur als Referenz übergeben wird.

```
#include <iostream>
#include <vector>
using namespace std;

double sum_below_limit(const vector<double> &vector, double limit)
{
    double summ = 0.0;
```

```
    for (int i : vector)
    {
        if (i < limit)
        {
            summ += i;
        }
    }
    return summ;
}
int main()
{
    vector<double> v = {1.0, 2.0, 3.0, 4.0, 5.0, 42.0};
    cout << sum_below_limit(v, 4.0) << "\n";
    return 0;
}
```

3 Vektoren II

```
#include <iostream>
#include <vector>
using namespace std;

double scalar_product(const vector<double> &vector1,
                      const vector<double> &vector2)
{
    double summ = 0;
    for (int i = 0; i < vector1.size(); i++)
    {
        summ += vector1[i] * vector2[i];
    }
    return summ;
}
int main()
{
    vector<double> v1 = {1.0, 2.0, 3.0, 4.0};
    vector<double> v2 = {1.0, -1.0, -1.0, 1.0};
    cout << scalar_product(v1, v2) << "\n";
    return 0;
}
```

4 Vektoren III

```
#include <iostream>
```

```cpp
#include <vector>
#include <algorithm> // Tipp: max_element könnte hilfreich sein
using namespace std;

vector<double> normalize_vector(const vector<double> &input_vec)
{
    if (input_vec.empty())
    {
        cout << "vector is empty!!!" << endl;
        return {};
    }

    // 1. Finde das Maximum im Vektor
    // (Falls der Vektor leer ist, gib einen leeren Vektor zurück)
    double max_val = *max_element(input_vec.begin(), input_vec.end());
    if (max_val == 0)
    {
        cout << "vctor is zero!!!" << endl;

        return {};
    }

    // 2. Erstelle einen neuen Vektor für die Ergebnisse
    vector<double> result;
    for (int i = 0; i < input_vec.size(); i++)
    {
        result.push_back(input_vec[i] / max_val);
    }

    // 3. Berechne die normalisierten Werte und füge sie 'result' hinzu
    return result;
}
int main()
{
    vector<double> v = {2.0, 5.0, 10.0, 4.0};
    // vector<double> v1 = {0.0, 0.0, 0.0};
    // vector<double> v2 = {};

    vector<double> normalized = normalize_vector(v);
    // Ausgabe der neuen Werte
    for (double val : normalized)
    {
        cout << val << " ";
    }
```

```
46      cout << endl;
47      return 0;
48  }
```